

# vS-Graphs: Tightly Coupling Visual SLAM and 3D Scene Graphs Exploiting Hierarchical Scene Understanding

Ali Tourani<sup>1</sup>, Saad Ejaz<sup>1</sup>, Hriday Bavle<sup>1</sup>, Miguel Fernandez-Cortizas<sup>1</sup>,  
David Morilla-Cabello<sup>2</sup>, Jose Luis Sanchez-Lopez<sup>1</sup>, and Holger Voos<sup>1</sup> \*†‡§

March 12, 2026

## Abstract

Current Visual Simultaneous Localization and Mapping (VSLAM) systems often struggle to create maps that are both semantically rich and easy to interpret. While incorporating semantic scene knowledge helps build richer maps with contextual associations among mapped objects, representing them in structured formats such as scene graphs has not been widely addressed, leading to complex map comprehension and limited scalability. This paper introduces vS-Graphs, a novel real-time VSLAM framework that integrates vision-based scene understanding with map reconstruction and comprehensible graph-based representation. The framework infers structural elements (*i.e.*, rooms and floors) from detected building components (*i.e.*, walls and ground surfaces) and incorporates them into optimizable 3D scene graphs. This solution enhances the reconstructed map’s semantic richness, comprehensibility, and localization accuracy. Extensive experiments on standard benchmarks and real-world datasets demonstrate that vS-Graphs achieves an average of **15.22%** accuracy gain across all tested datasets compared to state-of-the-art VSLAM methods. Furthermore, the proposed framework achieves environment-driven semantic entity detection accuracy comparable to that of precise LiDAR-based frameworks, using only visual features.

The code is publicly available at [https://github.com/snt-arg/visual\\_sgraphs](https://github.com/snt-arg/visual_sgraphs) and is actively being improved. Moreover, a web page with additional media and evaluation results is available at <https://snt-arg.github.io/vsgraphs-results/>.

## 1 Introduction

Robust environment understanding, a core foundation of robots’ situational awareness [1] in the context of Simultaneous Localization and Mapping (SLAM), relies heavily on sensor quality and modality. While diverse sensors, *e.g.*, Light Detection And Ranging (LiDAR) and cameras, have been employed in SLAM, vision sensors offer a cost-effective solution for rich map reconstruction, forming a distinct category titled Visual SLAM (VSLAM) [2]. Among vision sensors, RGB-D cameras provide complementary visual and depth cues, overcoming monocular limitations by producing dense point clouds that capture fine spatial

<sup>\*1</sup>Authors are with the Automation and Robotics Research Group, Interdisciplinary Centre for Security, Reliability, and Trust (SnT), University of Luxembourg, Luxembourg. Ali Tourani is also associated with the Institute for Advanced Studies (IAS), University of Luxembourg, Luxembourg. Holger Voos is also associated with the Faculty of Science, Technology, and Medicine, University of Luxembourg, Luxembourg. {ali.tourani, saad.ejaz, hriday.bavle, miguel.fernandez, joseluis.sanchezlopez, holger.voos}@uni.lu

<sup>†2</sup>Author is with the Instituto de Investigación en Ingeniería de Aragón (I3A), Universidad de Zaragoza, Spain. davidmc@unizar.es

<sup>‡\*</sup>This research was funded, in whole or part, by the Luxembourg National Research Fund (FNR), DEUS Project, ref. C22/IS/17387634/DEUS. It was also partially funded by the Institute of Advanced Studies (IAS) of the University of Luxembourg through an “Audacity” grant (project TRANSCEND).

<sup>§\*</sup>For the purpose of open access, and in fulfillment of the obligations arising from the grant agreement, the author has applied a Creative Commons Attribution 4.0 International (CC BY 4.0) license to any Author Accepted Manuscript version arising from this submission.

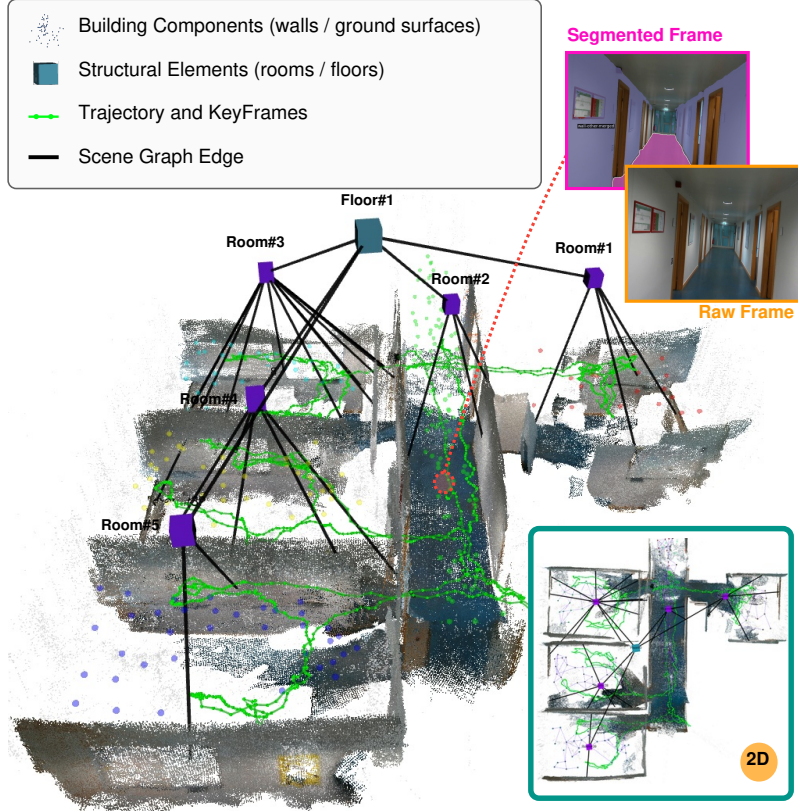


Figure 1: A reconstructed map with its corresponding optimizable 3D scene graph enriched with environment-driven semantic entities, generated by vS-Graphs (SMapper *MR03* sequence).

details, enabling precise detection, localization, and mapping of environmental elements [3]. To further enhance VSLAM performance, visual perception techniques are integrated, ranging from semantic scene understanding to the incorporation of artificial landmarks such as fiducial markers [4, 5].

Beyond enriching maps with visual and depth information, various methodologies aim to organize this data into interpretable and structured representations. Among them, *scene graphs* represent environments as hierarchical structures that capture the presence of objects, their attributes, and inter-relationships. These solutions provide a higher level of abstraction for scene understanding by outlining spatial associations among observed entities [6]. While scene graph-driven works like [7, 8] focus on tailoring geometric and semantic cues for reliable interpretation, approaches such as Hydra [9] and HOV-SG [10] serve as 3D scene graph builders (rather than SLAM systems), representing spatial hierarchies without estimating camera trajectories. Moreover, HOV-SG requires ground-truth semantics and an entire-environment map to construct the scene graph offline. In contrast, works like *S-Graphs* [11] push the boundaries by directly incorporating scene graphs into the SLAM pipeline, relying on LiDAR odometry with planar surface extraction within a unified optimization system, though without support for visual input.

Inspired by LiDAR-based *S-Graphs* [11], this paper proposes a VSLAM framework, **visual S-Graphs** (vS-Graphs), that tightly couples scene graph construction with VSLAM through joint optimization in a unified factor graph. vS-Graphs operates in real time and employs both visual and depth data to enhance map reconstruction and camera pose estimation. It reliably incorporates “building components” (*i.e.*, wall and ground surfaces), “structural elements” (*i.e.*, rooms and floors), and their spatial associations to produce a more geometry-aware and semantically coherent environment representation. Unlike LiDAR-based S-Graphs, which rely purely on geometric cues, vS-Graphs exploits the richer semantic and appearance information available in visual data to validate and contextualize structural elements. This not only strengthens category verification but also establishes a scalable foundation for incorporating additional components in future

extensions. The result is an optimizable, hierarchical 3D scene graph generated online alongside trajectory estimation, pairing robot poses from the underlying SLAM pipeline with recognized semantic entities, as depicted in Fig. 1. With this, the contributions of the paper are:

- A real-time multi-threaded VSLAM framework that generates optimizable 3D scene graphs during map reconstruction,
- A vision-based method for recognizing and mapping building components (*i.e.*, wall and ground surfaces), enhancing map richness and trajectory estimation,
- A mechanism for extracting high-level structural elements from the localized building components for advancing scene understanding; and
- Publicly available source code to facilitate reproducibility and further research in the field.

## 2 Related Works

Recent advances in computer vision and the development of reliable VSLAM frameworks, such as ORB-SLAM 3.0 [12], have enabled more robust localization and mapping systems. As noted by [13], depth data is essential for enhancing scene understanding and supporting downstream tasks such as environment modeling. In this regard, ElasticFusion [14] constructs globally consistent surfel maps for photometric tracking, but struggles with scalability and limited integration of environment-level cues. BAD SLAM [15] improves map and trajectory optimization via direct RGB-D Bundle Adjustment (BA), yet remains sensitive to initialization. More recent systems, like GS-SLAM [16], RTG-SLAM [17], and SplaTAM [18], incorporate 3D Gaussian representations to enhance tracking and map reconstruction. Nonetheless, they overlook the scene’s semantic context.

Building on these advances, RGB-D VSLAM frameworks have begun integrating semantic awareness into their mapping pipelines. Tools like Voxblox++ [19] augment online scanning with volumetric, object-centric mapping to improve recognition of scene elements. NICE-SLAM [20] and NIS-SLAM [21] combine neural implicit and hierarchical representations with pre-trained geometric priors to improve dense scene reconstruction at the cost of heavy computation and increased localization errors. Peng *et al.* [22] integrate Asymmetric Non-local Neural (ANN)-based semantic segmentation with weighted  $K$ -means depth clustering for static dense point cloud mapping, but their approach is not real-time, requiring an average of 100.8 ms per frame. DROID-SLAM [23] uses a fully differentiable pipeline to jointly refine camera poses and dense depth via Gauss–Newton updates driven by optical flow consistency, achieving strong accuracy at the expense of substantial memory and computational overhead. Methods such as OVD-SLAM [24] and YDD-SLAM [25] use Convolutional Neural Networks (CNNs) to filter feature points associated with specific semantic objects, thereby refining pose estimation and trajectory accuracy. Similarly, [26] introduces a binary CNN-based descriptor for robust feature matching and for improving initial pose measurements. SSF-SLAM [27] fuses semantic cues with scene-flow geometry via depth–density clustering to build 3D semantic point clouds, enabling improved scene understanding. Despite their pros, these systems remain limited to object-level reasoning rather than constructing comprehensive scene representations enriched with environment-driven entities.

Another research direction explores fiducial markers as reliable landmarks, offering an alternative to purely vision-based scene interpretation. Approaches such as [4,5] detect and map environment-driven entities labeled with markers, thereby contributing to a more comprehensive understanding of the environment layout. Nevertheless, their reliance on pre-placed markers restricts their applicability to controlled environments, limiting flexibility in unprepared or dynamic settings.

In contrast to prior semantic VSLAM systems that primarily emphasize object-level cues or dense reconstruction, vS-Graphs embeds scene graph construction directly into the VSLAM pipeline. By explicitly modeling environment-driven entities and their spatial relationships, it jointly optimizes camera poses and semantic structures, enabling coherent and interpretable indoor scene representations.

## 3 Proposed Method

### 3.1 System Overview

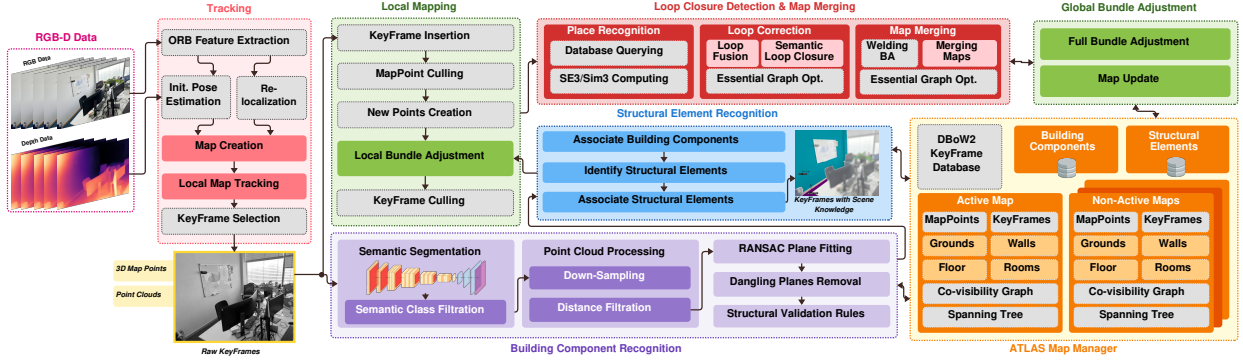


Figure 2: The multi-thread architecture of vS-Graphs. Modules with dashed borders and a light gray background are inherited directly from the baseline (*i.e.*, ORB-SLAM 3.0), while the remaining components are newly added or modified modules.

Building upon ORB-SLAM 3.0, vS-Graphs introduces substantial modifications to its baseline’s core modules and adds new threads for robust scene analysis and reconstruction. The system architecture, shown in Fig. 2, details the individual threads, components, and their interconnections. The current version supports RGB-D input, using depth data primarily to generate surface point clouds, which are then validated to support robust scene understanding. The core contribution lies in seamlessly integrating two novel threads: *Building Component Recognition* (§3.2) and *Structural Element Recognition* (§3.3). These threads are deeply embedded in vS-Graphs and work closely with other modules to enrich the reconstructed maps and ensure optimal performance.

In brief, RGB-D data is processed in real time, providing integrated visual and depth information to subsequent modules. Visual features are extracted and tracked across frames in the *Tracking* thread, where pose information is either initialized or refined, depending on the map reconstruction stage, thereby creating a 3D map of tracked features. KeyFrame selection, a critical step following feature extraction, is performed within the *Tracking* thread by analyzing visual cues. These KeyFrames contain 3D map points and point clouds, forming the foundation for subsequent processes. KeyFrames are then sent to the *Local Mapping* thread for map integration and optimization, with inaccurately posed KeyFrames culled for enhanced accuracy. Simultaneously, the *Building Component Recognition* thread identifies and localizes walls and ground surfaces by processing the KeyFrame-level visual-spatial data. They are detected by filtering the data through a CNN, followed by RANSAC plane fitting for global pose estimation. Concurrently, the *Structural Element Recognition* thread runs at fixed intervals, extracting higher-level entities, including rooms and floors, from the building components of the current (active) map. Eventually, the *local bundle adjustment* module in the *Local Mapping* thread jointly optimizes KeyFrame poses, map points, building components, and structural elements as coupled variables in the factor graph, ensuring that geometric and semantic entities are mutually refined for accurate map reconstruction.

At baseline, *Atlas* is the core of the multi-map management, maintaining mapped instances and their associations and supporting operations such as map merging. In vS-Graphs, we extend the module to store building components and structural elements, enriching map storage and retrieval beyond purely geometric information. Furthermore, the *Loop Closure Detection* checks for revisited locations and triggers *Global Bundle Adjustment* to optimize and merge the map upon detection of a loop. While it remains fundamentally geometric, the inclusion of environment-driven entities enables the system to verify and refine loop candidates. During global optimization, redundant building components and structural elements detected across overlapping KeyFrames are merged and re-optimized within the graph, with their associations updated to ensure map consistency. Although *semantic loop closure* provides complementary validation, fully semantic-driven loop closure remains an area for future investigation.

### 3.2 Building Component Recognition

This thread processes the point cloud and visual data from KeyFrames to extract fundamental environment-driven elements beneficial for scene understanding. The current version of vS-Graphs defines building components  $\Psi$  as walls and ground surfaces. Accordingly, each KeyFrame  $K = \{\mathbf{P}, L, \varepsilon\}$  passes through the *building component recognition* module  $B$ , generating identified structural element set  $\Pi_{\Psi}^K = B(\mathbf{K})$  for  $K$ . Here,  $\mathbf{P} = \{p_i \mid p \in \mathbb{R}^3, i \in \mathbb{N}\}$  is the point cloud,  $L$  is the matrix of RGB data, and  $\varepsilon$  represents additional metadata for mapping (such as camera pose).

Building component extraction is achieved by processing  $L$  with a semantic segmentation function  $S$ , which labels each pixel in the RGB data, retaining only relevant classes and discarding irrelevant ones. In contrast with the conventional object detection methods, panoptic scene segmentation provides *pixel-level class labels* and *instance differentiation*, enabling sharper boundary delineation and more precise object recognition. In principle, any reliable real-time panoptic segmentation framework can be integrated into vS-Graphs. However, the current version integrates Panoptic-FCN (pFCN) [28] and YOSO [29] by default, given their efficiency and strong balance between accuracy and real-time performance. In this regard,  $\hat{\mathbf{P}}_{\Psi}^K = S(\mathbf{P}, L)$  refers to the segmented visual-spatial data of  $K$ , where  $L_{(u,v)}$  is the semantic class label of a pixel at  $(u, v) \in \mathbb{N}^2$ . Applying semantic class filtration to filter building components' semantic classes  $\Psi$  (*i.e.*, wall and ground surfaces) takes place as follows:

$$\hat{\mathbf{P}}_{\psi}^K = \{p_j \mid p_j \in \mathbb{R}^3, L_{p_j} \in \Psi\} \quad (1)$$

$$\hat{\mathbf{P}}_{\Psi}^K = \{\hat{\mathbf{P}}_{\psi}^K\} = \{\hat{\mathbf{P}}_{wall}^K, \hat{\mathbf{P}}_{ground}^K\} \quad (2)$$

where  $\hat{\mathbf{P}}_{\Psi}^K \subset \mathbf{P}$  represents the set of semantically segmented point clouds, including the point clouds of  $\hat{\mathbf{P}}_{wall}^K$  and  $\hat{\mathbf{P}}_{ground}^K$ . It should be noted that the classification confidence  $\lambda_{(u,v)} \in \mathbb{R}$  for each pixel  $L_{\Psi(u,v)}$  is set for potential classification errors.

The next stage is to optimize  $\hat{\mathbf{P}}_{\Psi}^K$ , as it may contain noisy or low-resolution points that negatively impact subsequent steps. In this regard, each segmented visual-spatial point cloud  $\hat{\mathbf{P}}_{\psi}^K$  undergoes a two-stage preprocessing procedure. The first step is applying down-sampling to fetch  $\hat{\mathbf{P}}_{\psi_d}^K$ , a refined point cloud with reduced redundancy and noise, where  $\hat{\mathbf{P}}_{\psi_d}^K \subset \hat{\mathbf{P}}_{\psi}^K$ . As depth sensors exhibit increased noise at extreme ranges, the semantic classification of structural elements is most reliable within an optimal depth range. Thus, applying distance filtration based on the recommended sensor range results in retaining a subset of the point cloud as  $\hat{\mathbf{P}}_{\psi_{\zeta}}^K \subset \hat{\mathbf{P}}_{\psi_d}^K$ . The final processed point cloud, referred to as  $\hat{\mathbf{P}}_{\Psi_{\zeta}}^K = \{\hat{\mathbf{P}}_{\psi_{\zeta}}^K\}$ , is then forwarded to successive stages for further analysis.

Processing each  $\hat{\mathbf{P}}_{\psi_{\zeta}}^K$  through RANdom SAMple Consensus (RANSAC) [30] plane fitting algorithm results in detecting semantically-validated building components with their geometric equations. Thus, sets of random points  $\mathbf{p}_r \in \hat{\mathbf{P}}_{\psi_{\zeta}}^K$  are iteratively selected to calculate the normal vectors  $\mathbf{n} \in \mathbb{R}^3$  representing validated planar components  $\pi$  with the pre-defined distance  $d(\mathbf{p}_r, \pi) \leq \epsilon$ , where  $\epsilon$  is the inlier threshold. Accordingly, the final output representing all defined building components is as follows:

$$\Pi_{\psi}^K = \{\pi_{\psi_i}^K \mid \pi_{\psi_i}^K \in \mathbb{R}^3, i \in \mathbb{N}\} \quad (3)$$

$$\Pi_{\Psi}^K = \{\Pi_{\psi}^K\} = \{\Pi_{wall}^K, \Pi_{ground}^K\} \quad (4)$$

The remaining reliable elements are checked against structural validation rules  $\Pi_{\Psi}^K = V(\Pi_{\Psi}^K)$ , ensuring that they satisfy reasonable geometric constraints for the environment. For instance, walls should be represented as vertical planes, while ground surfaces should be defined as horizontal planes. The thread concludes the processing of KeyFrame  $K$  by storing its detected building components  $\Pi_{\Psi}^K$  within the current map in the *Atlas*, ensuring they contribute to the ongoing map reconstruction.

### 3.3 Structural Element Recognition

This thread repeatedly runs at constant time intervals (*i.e.*, every two seconds) to detect potential higher-level semantic entities that characterize the environment's layout, including rooms and floors. Structural elements comprise multiple building components, with their topological associations taken into account. The

thread actively searches for layouts that form rooms, where a room is a spatially enclosed area bounded by at least two walls that surround a confined free-space cluster on the ground plane. Additionally, a floor represents a higher-order structural element that encompasses a collection of rooms within a single building level.

In the first step, the map’s existing building components  $\mathbf{\Pi}_{\Psi} = \{\mathbf{\Pi}_{\Psi}^{K_i} \mid i \in \mathbb{N}\}$  are fetched from *Atlas*. Associating and merging redundant or conflicting building components is crucial to ensure consistency in the map reconstruction. In this context, the primary factors to assess are the spatial proximity and alignment of the mapped building components, requiring evaluating both the Euclidean distance and the angular difference between their normal vectors. Accordingly, the building component association condition is presented below:

$$\|\mathbf{\Pi}_{\Psi}^{K_i} - \mathbf{\Pi}_{\Psi}^{K_j}\| \leq \rho \quad \wedge \quad \cos^{-1} \left( \frac{\mathbf{n}_i \cdot \mathbf{n}_j}{\|\mathbf{n}_i\| \|\mathbf{n}_j\|} \right) \leq \eta \quad (5)$$

where  $\mathbf{n}$  is the normal vector of a particular surface and  $\rho$  and  $\eta$  refer to spatial proximity and angular alignment thresholds, respectively. The associated building components  $\mathbf{\Pi}_{\Psi}$  are subsequently utilized to detect structural elements  $\Delta_{\Phi}$ , defined as below:

$$\Delta_{\phi}^K = \{\delta_{\phi_m}^K \mid \delta_{\phi_m}^K \in \mathbb{R}^3, m \in \mathbb{N}\} \quad (6)$$

$$\Delta_{\Phi}^K = \{\Delta_{\phi}^K\} = \{\Delta_{room}^K, \Delta_{floor}^K\} \quad (7)$$

where  $\Delta_{\Phi}^K \in \Delta_{\Phi}$  is the set of structural elements belong to semantic classes  $\Phi$  found in KeyFrame  $K$ . The procedure for detecting structural elements is outlined below.

**Rooms.** A room  $\delta_r^K = \{\mathbf{\Pi}_{wall}, \pi_{ground}^{K_q}, \nu_r\}$  is a convex, spatially enclosed area bounded by at least two walls, each oriented toward a distinct free-space cluster  $\Upsilon = \{v_j \mid v_j \in \mathbb{R}^3, j \in \mathbb{N}\}$  in the global frame. The free-space cluster is calculated from the depth camera points using the Voxblox tool [31]. The associated set of walls is denoted as  $\mathbf{\Pi}_{wall} = \{\pi_{wall}^{K_1}, \dots, \pi_{wall}^{K_n}\}$ , where  $n \geq 2$ . Each wall  $\pi_{wall}^{K_i}$  is validated using *proximity* and *directionality* constraints, ensuring geometric consistency with its corresponding free-space. *Proximity* is calculated as:

$$\forall \pi_{wall}^{K_i} \in \mathbf{\Pi}_{wall}, \exists v_j \in \Upsilon : \left| \mathbf{n}_{wall}^{K_i \top} \cdot v_j + d_{wall}^{K_i} \right| \leq \omega \quad (8)$$

where  $\mathbf{n}_{wall}^{K_i} \in \mathbb{R}^3$  denotes the wall’s normal vector,  $d_{wall}^{K_i}$  is the plane offset, and  $\omega$  is a tunable threshold controlling the allowable distance between the wall and nearby free-space points. *Directionality* is computed as follows:

$$\forall \pi_{wall}^{K_i} \in \mathbf{\Pi}_{wall} : \mathbf{n}_{wall}^{K_i \top} (\nu_{\Upsilon} - \nu_{wall}^{K_i}) < 0 \quad (9)$$

where  $\nu_{wall}^{K_i} \in \mathbb{R}^3$  is the wall centroid and  $\nu_{\Upsilon} \in \mathbb{R}^3$  is the centroid of the associated free-space cluster  $\Upsilon$ . These formulations enable the detection of convex  $n$ -wall rooms with arbitrary wall orientations, beyond Manhattan layouts, by enforcing that adjacent walls face the enclosed cluster. The ground plane  $\pi_{ground}^{K_q}$  is associated with a room if it is approximately orthogonal to all wall normals and spatially enclosed by them. Accordingly:

$$\left| \mathbf{n}_{ground}^{\top} \cdot \mathbf{n}_{wall}^{K_i} \right| \leq \sin(\vartheta) \quad \wedge \quad d_{\pi_{wall}^{K_i}} \leq \nu_g \leq d_{\pi_{wall}^{K_j}} \quad (10)$$

where  $\vartheta$  defines the angular tolerance and  $\nu_g$  refers to the ground centroid lying between the nearest and farthest walls.

The optimization incorporates a geometry-aware constraint mechanism that updates each room’s centroid ( $\nu_{r_i}$ ) based on the centroids of its associated walls, while simultaneously identifying *parallel* and *perpendicular* wall pairs within the same room. Such patterns are naturally common in man-made environments, where walls are commonly constructed to align along dominant structural axes. These relationships introduce geometric constraints that minimize angular deviations, encouraging walls to align closer to  $0^\circ$  or  $90^\circ$ , respectively. In the absence of such relations, only the centroid consistency term remains active, maintaining spatial coherence without enforcing unnecessary structure. This flexible design enables the system to accommodate rooms of arbitrary or asymmetric geometry without relying on prior layout assumptions, unlike rectangular-shaped formulations commonly used in previous works [9, 10, 32]. In this context, if a room

contains parallel walls facing each other toward  $\nu_r$ , a parallelism constraint penalizes deviation through a cost given by the angle between their normals:

$$c_{\delta_r^K}^{\parallel}(i,j) = 1 - |\mathbf{n}_i^{\top} \cdot \mathbf{n}_j|, \quad \text{where } |\mathbf{n}_i| = |\mathbf{n}_j| = 1. \quad (11)$$

where  $\mathbf{n}_i$  and  $\mathbf{n}_j$  are the normalized values of normal vectors of the two wall planes. The cost reaches zero for perfectly parallel walls ( $\mathbf{n}_i \parallel \mathbf{n}_j$ ) and increases proportionally with angular deviation, providing a numerically stable measure of parallelism. Similarly, perpendicularity is calculated as:

$$c_{\delta_r^K}^{\perp}(i,j) = |\mathbf{n}_i^{\top} \cdot \mathbf{n}_j|, \quad \text{where } |\mathbf{n}_i| = |\mathbf{n}_j| = 1. \quad (12)$$

Finally, a centroid consistency term  $c_{\nu_r}$  keeps the room centroid  $\nu_r$  aligned with the mean of its wall centroids. The overall cost function for room-level constraints is:

$$c_{\nu_r} = \|\hat{\nu}_r - \frac{1}{n} \sum_{j=1}^n \nu_{wall}^{K_j}\|^2 \quad (13)$$

$$c_{\delta_r^K} = \frac{1}{N_{\parallel}} \sum_{i,j=0}^{N_{\parallel}} c_{\delta_r^K}^{\parallel}(i,j) + \frac{1}{N_{\perp}} \sum_{i,j=0}^{N_{\perp}} c_{\delta_r^K}^{\perp}(i,j) + c_{\nu_{r_i}} \quad (14)$$

where  $N_{\parallel}$  and  $N_{\perp}$  denote the number of parallel and perpendicular walls. Hence, when no parallel or perpendicular relations are detected, only the Euclidean-norm-based centroid update is activated, preserving localization accuracy while enhancing the structural coherence of the 3D scene graph. The factor weights were determined empirically by evaluating different configurations on a subset of sequences and selecting values that balance structural regularization with reprojection accuracy. These weights are fixed across all datasets and sequences without per-dataset tuning. All weight values are provided in the open-source implementation to facilitate reproducibility.

**Floors.** A floor  $\delta_f^K = \{\Delta_r, \nu_f, \pi_{floor}^{K_i}\}$  is defined by a set of rooms  $\Delta_r$ , all sharing a common horizontal reference plane  $\pi_{floor}^{K_i}$  and the floor centroid  $\nu_f$ . The floor plane is associated with the constituent rooms if it satisfies the co-planarity condition within a vertical tolerance. In practice, this validation step is ignored in the current vS-Graphs implementation due to its single-floor design constraint. Additionally, its spatial extent is bounded by the union of all constituent room boundaries. The floor centroid  $\nu_f$  is computed as the weighted arithmetic mean of all constituent room centroids:

$$\nu_f = \frac{\sum_{j=1}^{N_r} \nu_{r_j}}{N_r} \quad (15)$$

where  $N_r = |\Delta_{room}|$ . The cost function to optimize the floor's vertex node is calculated as follows:

$$c_{\delta_f^K} = \sum_{t=1}^T \|\hat{\nu}_f - h(\Delta_r^K)\|_{\Lambda_{\delta_f,t}}^2 \quad (16)$$

where  $h(\dots)$  is a hierarchical mapping function that associates the floor's structural components with its centroid.

### 3.4 Scene Graph Structure

Fig. 3 illustrates the 3D scene graph structure generated using vS-Graphs, along with its corresponding modules shown in Fig. 2. In contrast to the traditional SLAM reconstructed maps, the geometric replicas of vS-Graphs are augmented with hierarchical, rich semantic data, enabling meaningful interaction with the environment. The generated scene graph fills the contextual understanding gap and provides better scene interpretation, enabling complementary missions such as scalable map-building and improved decision-making.

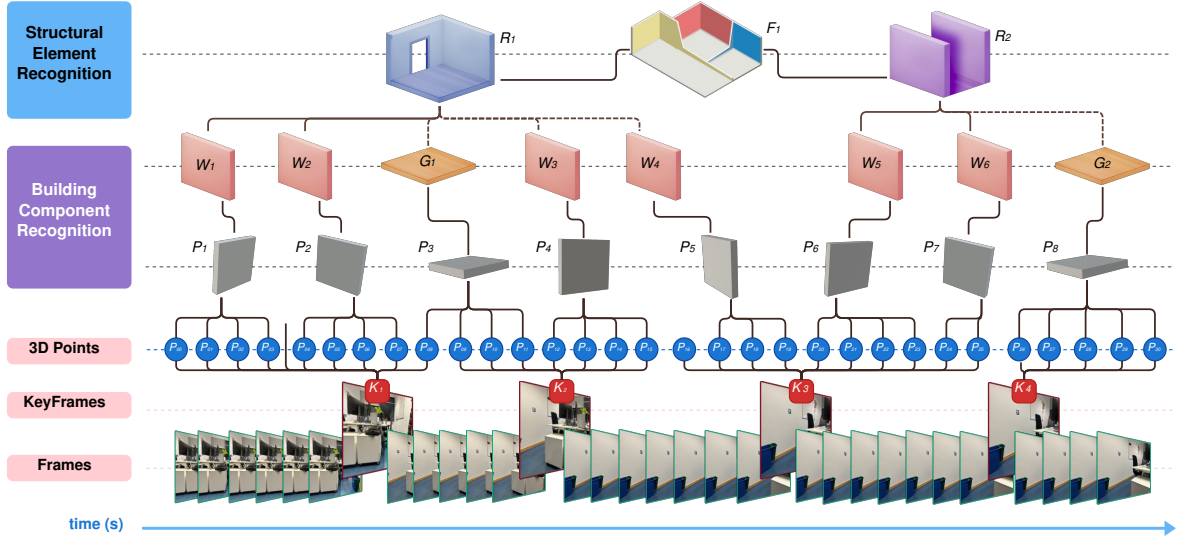


Figure 3: Scene graph structure generated using vS-Graphs, creating a hierarchical representation of the environment.

Table 1: Characteristics of the *AutoSense* in-house dataset. Each sequence includes one ground and one floor surface by design. *BC* denotes building components (*ground/walls*), and *SE* denotes structural elements (*rooms/floor*).

| Seq. | #BC<br>(Ground+ Walls) | #SE<br>(Room+ Floor) | Description                          |
|------|------------------------|----------------------|--------------------------------------|
| SR01 | 4 (1+3)                | 2 (1+1)              | partial view of a room               |
| SR02 | 4 (1+3)                | 2 (1+1)              | room corner with intersecting walls  |
| SR03 | 6 (1+5)                | 2 (1+1)              | a single rectangular room            |
| MR01 | 14 (1+13)              | 4 (3+1)              | across rooms connected to a corridor |
| MR02 | 13 (1+12)              | 4 (3+1)              | adjoining rooms linked by a corridor |
| MR03 | 22 (1+21)              | 6 (5+1)              | a suite of interconnected rooms      |

## 4 Experimental Results

### 4.1 Evaluation Criteria

**Setup.** Evaluations were conducted on a system with an Intel® Core™ i9-11950H processor (2.60 GHz), a 4GB NVIDIA T600 Mobile GPU, and 32GB of RAM.

**Datasets.** We evaluated vS-Graphs on standard benchmarks and in-house datasets to assess its performance across diverse environments and scene complexities. The standard datasets include ICL [33] photorealistic synthetic scenes and real-world data from OpenLORIS [34], ScanNet [35], and TUM-RGBD [36]. The in-house dataset, SMapper, was collected using a custom-built device introduced in [37] that records RGB-D video and LiDAR point clouds. It features diverse real-world indoor environments with varied architectural layouts and multi-room scenarios. The ground-truth data in the dataset were derived from reliable LiDAR poses and point clouds generated by *S-Graphs*. Table 1 outlines the dataset characteristics, encompassing approximately 44 minutes of operation and 620 meters of trajectories across diverse scenarios. Additionally, Fig. 4 showcases the device and sample sequences. Due to space constraints, the complete evaluation results and figures are available at <https://snt-arg.github.io/vsgraphs-results/>.

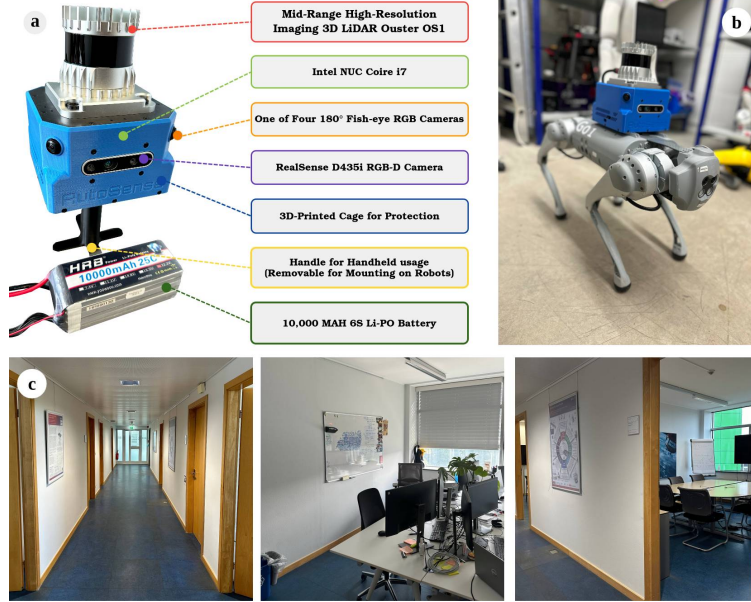


Figure 4: In-house dataset collection using the *SMapper* device: **a)** the setup overview, **b)** the device mounted on a legged robot, and **c)** some instances of the collected data.

## 4.2 Evaluations and Discussions

### 4.2.1 Trajectory Estimation Performance

To assess trajectory estimation accuracy, vS-Graphs was benchmarked against established and robust VSLAM systems, including ORB-SLAM 3.0 (baseline) [12], DROID-SLAM [23], ElasticFusion [14], and BAD SLAM [15]. Marker-based methods (*e.g.*, [4]) were excluded due to their reliance on artificial landmarks and external pose constraints, which limit their applicability in marker-free environments. Similarly, neural field-based approaches (such as [20]) were omitted because they rely on learned scene priors rather than explicit geometric-semantic representations. Additionally, vS-Graphs was evaluated using different segmentation backbones (pFCN and YOSO) to assess the effects of segmentation quality and efficiency on trajectory robustness.

Table 2 presents the evaluation results, where each system is evaluated over eight runs per sequence, and performance is measured using Absolute Trajectory Error (ATE) reported in centimeters. Accordingly, vS-Graphs variants consistently achieve state-of-the-art performance, ranking first or second in nearly all evaluations across segmentation backbones and module configurations. This improvement stems from the additional geometric and semantic constraints imposed by the accurate localization of building components and structural elements. When combined with loop closures, these constraints yield even larger gains, *e.g.*, 75.38% for “*deer-w*” with the YOSO-based configuration. However, rapid motion and noisy depth data can negatively affect performance (*e.g.*, “*SR01*”, BC-only). DROID-SLAM performs better in rapid-motion scenarios (TUM-RGBD instances), but at the cost of substantially higher GPU memory usage and limited real-time capability. Another notable observation is that incorporating room entities further improves trajectory accuracy, particularly when room-wall constraints are enforced. This effect is most evident in looped (“*deer-w*”) and multi-room environments (“*MR01*”), where maintaining geometric consistency is crucial, reducing ATE by 13.82% and 15.22% w.r.t. the baseline. It can be seen that the choice of panoptic backbone (YOSO or pFCN) has only a marginal effect on the trajectory accuracy of vS-Graphs, with higher segmentation quality naturally yielding more precise structural mapping.

It should be noted that a primary challenge shared by all tested VSLAMs arises in low-texture scenes (*e.g.*, corridors with uniformly painted walls), negatively impacting feature-matching and tracking procedures. In vS-Graphs, this can affect pose estimation and localization of building components and propagate the error through the structural element recognition stage. Additionally, treating building components as planar

Table 2: Absolute Trajectory Error (ATE) in centimeters (cm) for various VSLAM algorithms across eight iterations. The best and second-best results are **boldfaced** and underlined, respectively. vS-Graphs is evaluated with multiple semantic segmentation backbones (YOSO and pFCN) and with different levels of entity integration, including only building component cues (*vS-Graphs BC*) and the full pipeline with structural element recognition (*vS-Graphs*). The minimum completion rate of each sequence is set to 50%, and dashes indicate unavailable data due to tracking failures. Additionally, the “%Gain” rows quantify the percentage improvement achieved by vS-Graphs compared to its baseline (ORB-SLAM 3.0).

| Dataset              | Sequence      | Time (s) | Length (m) | Methodology |          |               |             |              |             |              |              |         |        |
|----------------------|---------------|----------|------------|-------------|----------|---------------|-------------|--------------|-------------|--------------|--------------|---------|--------|
|                      |               |          |            | ORB-SLAM3   |          |               |             | vS-Graphs    |             |              |              | %Gain   |        |
|                      |               |          |            | ORB-SLAM3   | BAD SLAM | ElasticFusion | DROID-SLAM  | BC Only      |             | BC+SE        |              | BC Only | BC+SE  |
|                      |               |          |            |             |          |               |             | YOSO pFCN    | YOSO pFCN   | YOSO pFCN    | YOSO pFCN    |         |        |
| ICL                  | deer-ground   | 79.9     | 11.7       | <b>0.74</b> | 147.60   | 14.50         | 0.90        | 0.81         | 0.91        | <u>0.78</u>  | 0.89         | -8.89   | -23.40 |
|                      | deer-walkh    | 65.3     | 34.6       | 6.96        | —        | 82.50         | 44.64       | 6.08         | 6.23        | <b>5.96</b>  | 6.14         | 12.64   | 10.54  |
|                      | deer-walk     | 64.0     | 45.1       | 9.91        | 147.40   | 62.00         | 2.87        | 8.82         | 12.19       | <b>2.44</b>  | <u>4.02</u>  | 10.97   | -22.98 |
|                      | deer-running  | 28.4     | 16.0       | 6.92        | —        | 78.70         | —           | 5.34         | 5.47        | <b>2.83</b>  | <u>3.34</u>  | 22.82   | 20.98  |
|                      | deer-mav-fast | 102.5    | 74.9       | 2.67        | 4.60     | —             | 10.39       | 2.65         | <u>2.07</u> | 2.23         | <b>1.26</b>  | 0.96    | 22.57  |
|                      | <b>Total</b>  | 340      | 182.3      | 5.44        | 99.87    | 59.43         | 14.70       | 4.74         | 5.37        | <b>2.85</b>  | <u>3.13</u>  | 7.70    | 1.54   |
| OpenLORIS            | office1-1     | 27.0     | 5.3        | <u>6.37</u> | 12.30    | 6.88          | 8.65        | 6.41         | <b>6.33</b> | 6.42         | <u>6.37</u>  | -0.66   | 0.50   |
|                      | office1-2     | 30.2     | 5.9        | 10.05       | 14.00    | 11.26         | 11.12       | 9.91         | 9.93        | <b>9.28</b>  | <u>9.68</u>  | 1.39    | 1.24   |
|                      | office1-3     | 12.1     | 0.4        | 14.73       | 19.50    | —             | 16.75       | 14.57        | 16.05       | <b>14.44</b> | <u>15.01</u> | 1.08    | -9.00  |
|                      | office1-4     | 29.4     | 6.1        | 12.51       | 34.30    | 18.03         | 16.96       | 12.68        | 12.55       | <u>11.54</u> | <b>11.36</b> | -1.37   | -0.33  |
|                      | office1-5     | 52.6     | 10.5       | 11.57       | 30.20    | 22.15         | 24.94       | 11.11        | 11.29       | <u>10.71</u> | <b>10.59</b> | 4.03    | 2.48   |
|                      | office1-6     | 36.5     | 5.5        | 5.86        | 10.70    | —             | 8.14        | 5.90         | 5.89        | <u>5.70</u>  | <b>5.67</b>  | -0.68   | -0.45  |
|                      | office1-7     | 38.6     | 6.1        | 17.17       | 20.30    | —             | 11.28       | 17.55        | 17.50       | <b>13.00</b> | <u>13.25</u> | -2.17   | -1.92  |
|                      | <b>Total</b>  | 226.4    | 39.8       | 11.18       | 20.19    | 14.58         | 15.61       | 11.16        | 11.36       | <b>10.16</b> | <u>10.28</u> | 0.23    | -1.07  |
| ScanNet              | scn0041.01    | 75.5     | 25.5       | 14.26       | 22.20    | 21.96         | 14.87       | 14.29        | 14.06       | <b>13.56</b> | <u>13.78</u> | -0.27   | 1.37   |
|                      | scn0200.00    | 37.3     | 5.9        | 4.88        | —        | —             | —           | 4.67         | 4.65        | <u>4.55</u>  | <b>4.42</b>  | 4.36    | 4.79   |
|                      | scn0614.01    | 36.2     | 13.3       | 13.55       | 13.90    | 20.23         | 12.37       | 12.50        | 13.38       | <b>12.36</b> | <u>12.45</u> | 7.75    | 1.26   |
|                      | scn0626.00    | 21.1     | 6.1        | 9.60        | 27.30    | 18.91         | 8.49        | 9.50         | 9.22        | <u>8.73</u>  | <b>8.71</b>  | 1.05    | 4.02   |
|                      | <b>Total</b>  | 170.1    | 50.8       | 10.57       | 21.13    | 20.36         | 11.91       | 10.24        | 10.33       | <b>9.80</b>  | <u>9.84</u>  | 3.22    | 2.86   |
| TUM-RGBD             | frb1-desk     | 23.8     | 9.3        | 2.07        | 2.20     | 2.55          | <b>1.80</b> | 1.98         | <u>1.96</u> | 2.04         | 2.03         | 4.25    | 5.18   |
|                      | frb1-desk2    | 25.1     | 9.9        | 3.23        | 2.90     | 7.87          | <b>2.46</b> | 3.19         | 3.19        | 2.89         | <u>2.85</u>  | 1.17    | 1.21   |
|                      | frb1-room     | 49.1     | 16.0       | 13.25       | 20.90    | 16.75         | <b>3.76</b> | 13.02        | 13.29       | <u>5.13</u>  | 6.77         | 1.75    | -0.26  |
|                      | frb2-desk-prs | 142.1    | 18.8       | 1.90        | 9.60     | 4.51          | 6.22        | <u>1.79</u>  | <b>1.78</b> | <u>1.79</u>  | 1.81         | 5.69    | 6.07   |
|                      | frb3-strct    | 31.9     | 6.0        | 1.62        | 2.10     | 1.75          | 1.74        | 1.54         | <u>1.53</u> | <b>1.52</b>  | 1.54         | 5.04    | 5.64   |
|                      | <b>Total</b>  | 272      | 50.7       | 4.41        | 7.54     | 6.68          | 3.20        | 4.31         | 4.35        | <b>2.67</b>  | <u>3.00</u>  | 3.58    | 3.57   |
| SMapper              | SR01          | 61.6     | 8.5        | 7.98        | 13.00    | <b>6.62</b>   | 8.03        | 8.10         | 8.16        | <u>7.34</u>  | 7.63         | -1.58   | -2.28  |
|                      | SR02          | 80.4     | 11.9       | 8.75        | 13.50    | —             | 9.38        | <b>8.21</b>  | 8.29        | <u>8.23</u>  | 8.26         | 6.15    | 5.32   |
|                      | SR03          | 170.4    | 36.2       | 10.90       | 55.70    | —             | 12.70       | <u>10.70</u> | 10.71       | <b>10.51</b> | 11.04        | 1.85    | 1.71   |
|                      | MR01          | 305.7    | 71.8       | 15.26       | 41.30    | —             | 14.88       | 15.44        | 14.98       | <b>12.23</b> | <u>12.75</u> | -1.13   | 1.85   |
|                      | MR02          | 210.2    | 51.7       | 23.97       | —        | 53.48         | 2406        | 21.68        | 23.78       | <b>20.13</b> | <u>21.19</u> | 9.58    | 0.83   |
|                      | MR03          | 783.3    | 135.5      | 22.19       | 81.21    | —             | 19.03       | 22.00        | 19.23       | <b>17.70</b> | <u>17.84</u> | 0.87    | 13.35  |
|                      | <b>Total</b>  | 1611.6   | 295.2      | 14.84       | 40.94    | 30.05         | 16.01       | 14.35        | 14.19       | <b>12.69</b> | <u>13.12</u> | 2.63    | 3.46   |
| <b>Overall Total</b> |               | 43m 40s  | 618.8m     | 9.29        | 37.93    | 26.22         | 12.29       | 8.96         | 9.12        | <b>7.63</b>  | <u>7.87</u>  | 3.47    | 2.07   |

surfaces might limit the framework’s applicability in environments with irregular geometries (*e.g.*, curved walls). Furthermore, overly or loosely permissive association thresholds can degrade building component identification performance, making careful threshold selection crucial.

#### 4.2.2 Mapping Performance

Additionally, analyzing the accuracy of the reconstructed maps against SMapper’s ground truth data revealed that *vS-Graphs* performs more robustly compared to ORB-SLAM 3.0 in terms of Root Mean Square Error (RMSE). As shown in Fig. 5, the median RMSE is consistently lower in *vS-Graphs*, indicating a higher level of overall mapping precision. *vS-Graphs* achieves superior mapping accuracy despite generating maps with  $\sim 10.15\%$  fewer points on average than the baseline, owing to its environment-driven constraints that enable a more coherent reconstruction.

#### 4.2.3 Scene Understanding Performance

This experiment evaluates vS-Graphs in terms of semantic scene understanding, with a particular focus on detecting entities essential for interpreting the environment’s structural layout. For benchmarking, we selected the multi-room sequences from the *AutoSense* dataset, which include ground truth annotations derived from LiDAR scans. Table 3 reports a quantitative comparison of vS-Graphs against two state-of-the-art approaches: Hydra [9] and *S-Graphs* [11]. For presentation clarity, *precision* and *recall* values for

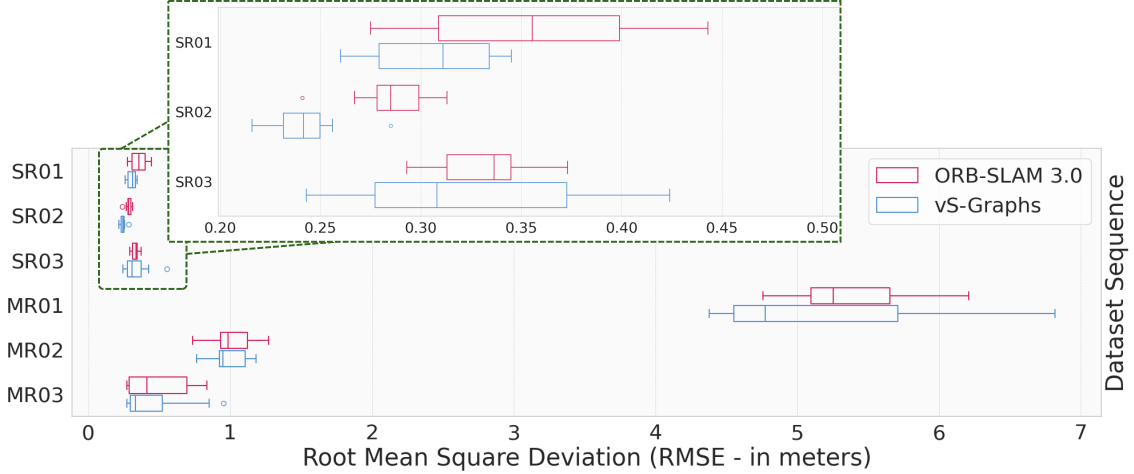


Figure 5: Mapping performance across eight iterations, showing SMapper sequences with less than one meter RMSE.

structural elements were omitted from the table to avoid visual saturation. The ground truth counts of rooms and walls were obtained by manual inspection of the dataset, where each entity was visually verified and counted. While *S-Graphs* benefits from the geometric accuracy of LiDAR point clouds, Hydra was configured to use visual point clouds, ensuring a fair comparison against our purely vision-based approach.

Experimental results show that vS-Graphs, despite operating solely on RGB-D input, achieves an accuracy comparable to the LiDAR-based method (*S-Graphs*) in detecting building components and structural elements. This highlights the effectiveness of our visual feature processing and scene graph generation in capturing structural layouts with high precision. Notably, vS-Graphs demonstrates higher *recall*, even in larger and more complex environments such as MR03, owing to its visual verification of building components that ensures consistent entity detection across extended scenes. It should be noted that “wall” entities are not directly provided in Hydra; therefore, Hydra’s performance is assessed based on correct “room” element counting and recognition. Additionally, Fig. 6 provides a qualitative comparison of the reconstructed scene graphs generated by vS-Graphs, *S-Graphs*, and Hydra across two representative dataset sequences.

Table 3: Scene understanding accuracy of vS-Graphs on multi-room sequences of SMapper. *BC* and *SE* denote “building components” and “structural elements,” respectively, while *GT* indicates the ground truth count of manually verified items.

|                                   |          | Detected / GT |       | Precision | Recall |
|-----------------------------------|----------|---------------|-------|-----------|--------|
| Method                            | Sequence | BC            | SE    | BC        | BC     |
| <b><i>S-Graphs</i></b><br>[11]    | MR01     | 11 / 14       | 4 / 4 | 0.92      | 0.92   |
|                                   | MR02     | 12 / 13       | 4 / 4 | 1.00      | 0.92   |
|                                   | MR03     | 20 / 22       | 6 / 6 | 0.90      | 0.95   |
| <b>Hydra</b><br>[9]               | MR01     | N/A           | 4 / 4 | N/A       | N/A    |
|                                   | MR02     | N/A           | 5 / 4 | N/A       | N/A    |
|                                   | MR03     | N/A           | 6 / 6 | N/A       | N/A    |
| <b><i>vS-Graphs</i></b><br>(ours) | MR01     | 13 / 14       | 4 / 4 | 0.86      | 1.00   |
|                                   | MR02     | 13 / 13       | 4 / 4 | 0.92      | 0.92   |
|                                   | MR03     | 23 / 22       | 6 / 6 | 0.96      | 1.00   |

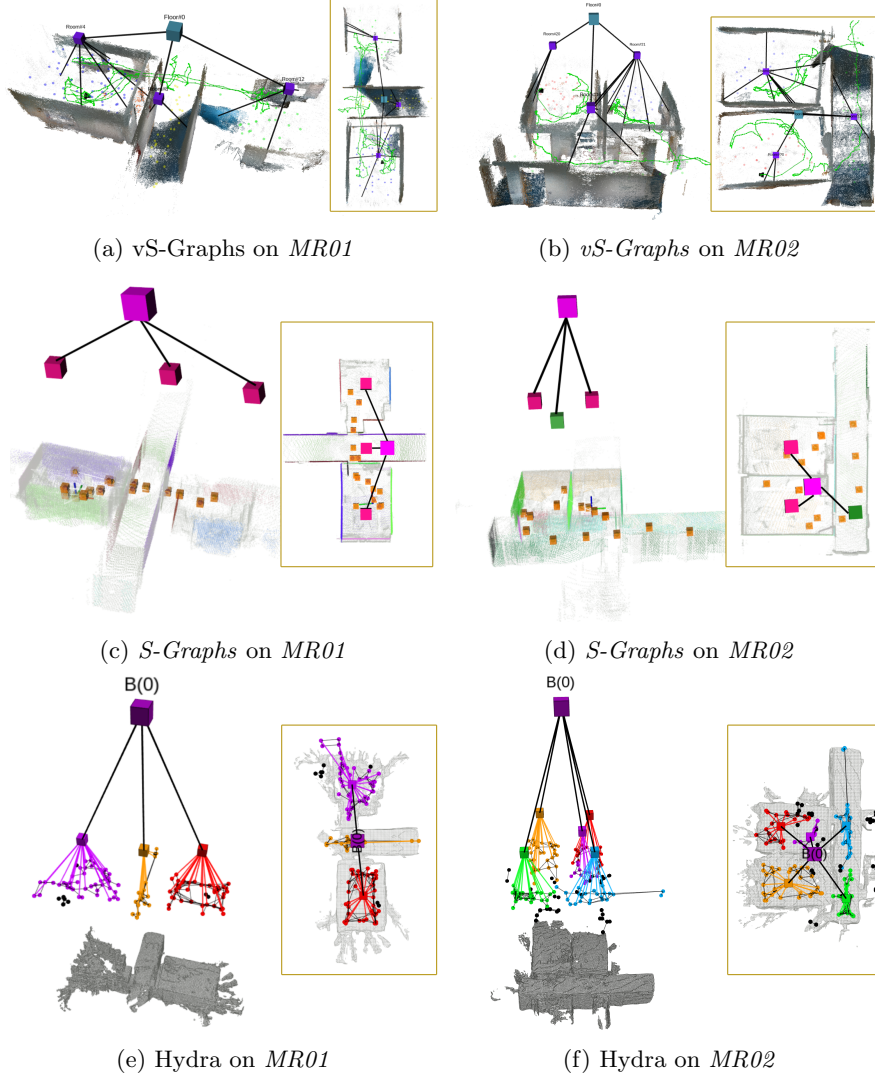


Figure 6: Qualitative comparison of reconstructed scene graphs on multi-room sequences of the *AutoSense* dataset.

#### 4.2.4 Runtime Analysis

vS-Graphs achieves real-time performance with an average processing rate of  $22 \pm 3$  FPS, exceeding the 20 FPS threshold considered for real-time operation. This is accomplished through a multi-threaded architecture, as shown in Fig. 7. The “*Tracking*” thread processes visual features at the frame level, while “*Local Mapping*” concurrently maps objects and optimizes their positions. “*Building Component Recognition*”, running in parallel at the KeyFrame level, identifies potential wall and ground surfaces from the online panoptic segmentation. The “*Structural Element Recognition*” runs less frequently and in constant periods (every two seconds) to infer rooms and floors in the map. Compared to ORB-SLAM 3.0’s  $29 \pm 3$  FPS on the same hardware and dataset, the slightly reduced frame rate is a reasonable trade-off for vS-Graphs’s rich semantic scene understanding.

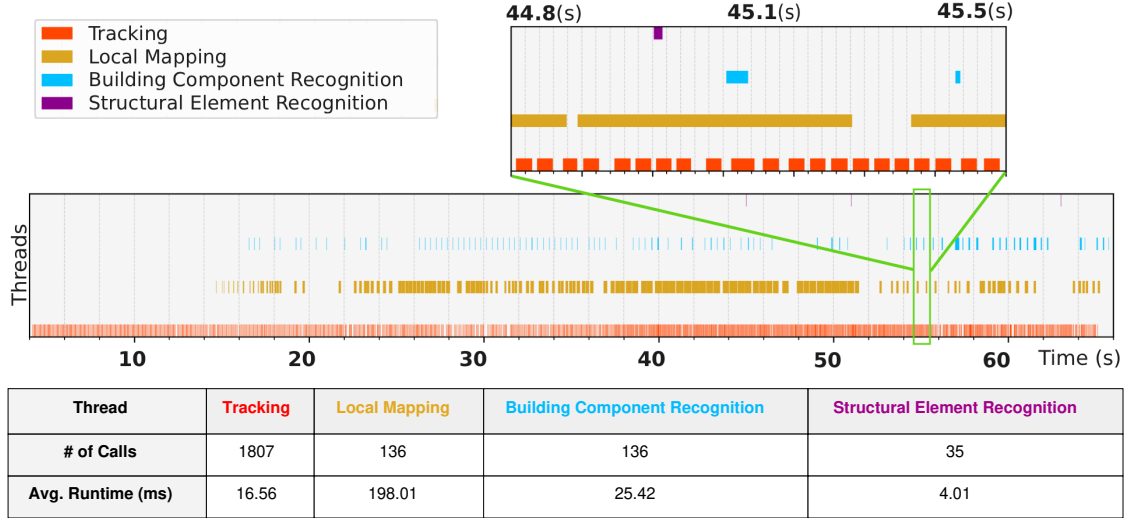


Figure 7: Timeline of thread execution within vS-Graphs while processing a sample dataset instance (sequence *SR01*).

## 5 Conclusions

This paper introduced vS-Graphs, a real-time VSLAM framework that reconstructs the robot’s operating environment using optimizable hierarchical 3D scene graphs. It detects building components (wall and ground surfaces), from which structural elements (rooms and floors) are inferred, and incorporates them into hierarchical representations. Consequently, beyond enhancing map reconstruction by integrating these entities, vS-Graphs offers structured representations of spatial relationships among high-level environment-driven semantic entities. Experimental results on standard and in-house indoor datasets demonstrated that the framework achieves superior trajectory estimation and mapping performance compared to state-of-the-art VSLAMs, reducing trajectory error by 15.22% across a wide range of dataset instances. Other evaluations have shown that the visual features processed by vS-Graphs can identify semantic entities describing the environment’s layout with accuracy comparable to that of precise LiDAR-based methods.

Future work includes integrating additional building components (*e.g.*, ceilings, doorways), extending support for complex and concave room layouts via GNN-based approaches, and developing fully semantic-driven loop closure detection, where revisited areas can be identified primarily based on previously detected structural elements and building components.

## Appendix I. Semantic Augmentation using Fiducial Markers

In vS-Graphs, fiducial markers serve as an **optional semantic augmentation mechanism**, providing high-level contextual information to the reconstructed map. Unlike marker-based Visual SLAM methods [4, 5, 38, 39], where markers play a central role in localization and map construction, in our vS-Graphs system, their role is supportive rather than structural. The pipeline does not depend on the presence of markers for map accuracy or robustness; instead, they enrich the representation by linking structural elements to semantic labels. To enable this, we assume that each structural element (*e.g.*, a room) may contain one unique **ArUco marker**. This criterion is implemented during the data collection procedures, where unique markers are placed in each free-space zone, such as rooms or corridors, and their associations are stored in a dedicated environment-driven database (JSON file). For instance, a marker with `marker_id = 5` is assigned to “*corridor-A*”, while marker `marker_id = 8` was assigned to “*office-B*”. At runtime, if (i) **the ArUco detector is active** and (ii) **the corresponding association file (database of marker information) is provided**, the system uses this information to enrich detected structural elements semantically. Thus, if the pose of a detected marker falls within the area of a recently detected n-wall room, it will be augmented with the semantic label stored in the database.

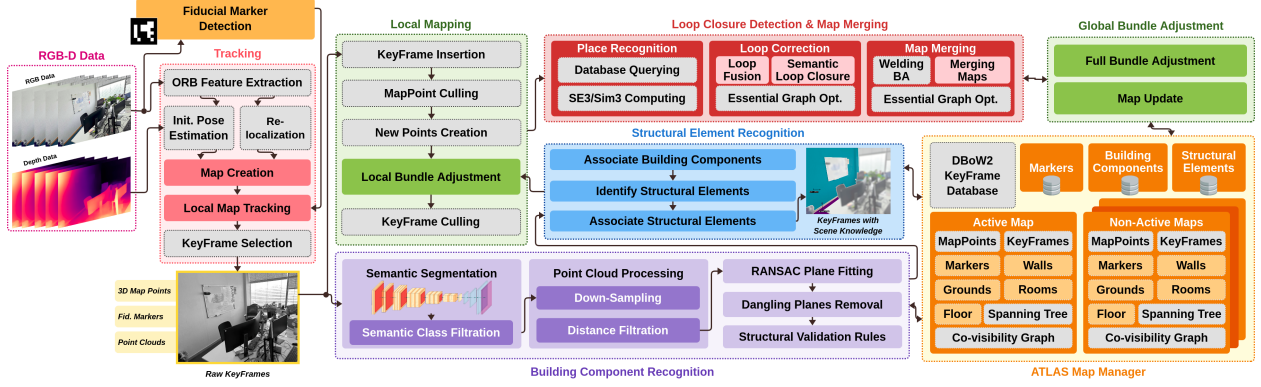


Figure 8: The multi-thread architecture of vS-Graphs with the optional fiducial marker detector module. Modules with dashed borders and a light gray background are inherited directly from the baseline (*i.e.*, ORB-SLAM 3.0), while the remaining components are the contributions of vS-Graphs.

Fig. 8 shows the overall architecture of vS-Graphs, including the fiducial marker detection and integration module, where recognized markers are stored in the *ATLAS Map Manager*. In brief, RGB-D data is processed in real time, providing integrated visual and depth information to subsequent modules. Meanwhile, “*Fiducial Marker Detection*” (utilizing the ArUco library in this work) operates independently on input frames, identifying markers *only if they are present* in the environment and associating them with the corresponding frames. Visual features are extracted and tracked across sequential frames in the “*Tracking*” thread, where pose information is either initialized or refined, depending on the map reconstruction stage, creating a 3D map with tracked features across frames. KeyFrame selection, a critical step following feature extraction, is performed within the “*Tracking*” thread by analyzing the visual data. These KeyFrames contain 3D map points, point clouds, and potentially detected fiducial markers, forming the foundation for subsequent processes.

Accordingly, each fiducial marker  $\mathbf{m}_i^K = \{\mathbf{p}_m, \nu_m\}$  in  $G_t$  is constrained by the KeyFrame  $K$  observing it, where  $\mathbf{p}_m \in SE(3)$  is the marker’s pose and  $\nu_m$  is its center point. The mentioned constraint is defined below:

$$\sigma_m({}^G\mathbf{K}, \mathbf{m}_i^K) = \|{}^L\mathbf{m}_i \boxplus {}^G\mathbf{K} \boxminus {}^G\mathbf{m}_{i1}\|_{\Lambda_{\tilde{\mathbf{m}}_i}}^2 \quad (17)$$

where  ${}^L\mathbf{m}_i$  refers to the locally observed marker’s pose,  $\boxplus$  and  $\boxminus$  represent the composition and inverse composition,  $\|\dots\|$  is the *Mahalanobis* distance, and  $\Lambda_{\tilde{\mathbf{m}}_i}$  is marker’s information matrix.

The marker  $\mathbf{m}_i^{K_j} \in \mathbf{M}$  is associated with a structural element if it lies within the spatial bounds of the detected room/corridor  $\delta^{K_g} \in \Delta_\phi$ . Thus, the Euclidean distance between the center point  $\nu_m$  of  $\mathbf{m}_i^{K_j}$  and the room/corridor’s centroid  $\nu_s$ , expressed as  $d(\nu_m, \nu_s) \leq \epsilon_s$ , where  $\epsilon_s$  is the proximity threshold. The supporting condition is that  $\nu_m$  must be enclosed among all bounding walls  $\Pi_{wall}$  with the normal vector  $\mathbf{n}$ :

$$\forall \pi_{wall}^{K_i} \in \Pi_{wall}, \quad (\mathbf{n}_{wall}^{K_i} \cdot (\nu_m - \nu_r)) \leq 0 \quad (18)$$

With this, Fig. 9 illustrates the extended scene graph structure of the vS-Graphs framework, emphasizing the role of fiducial markers in labeling the detected structural elements.

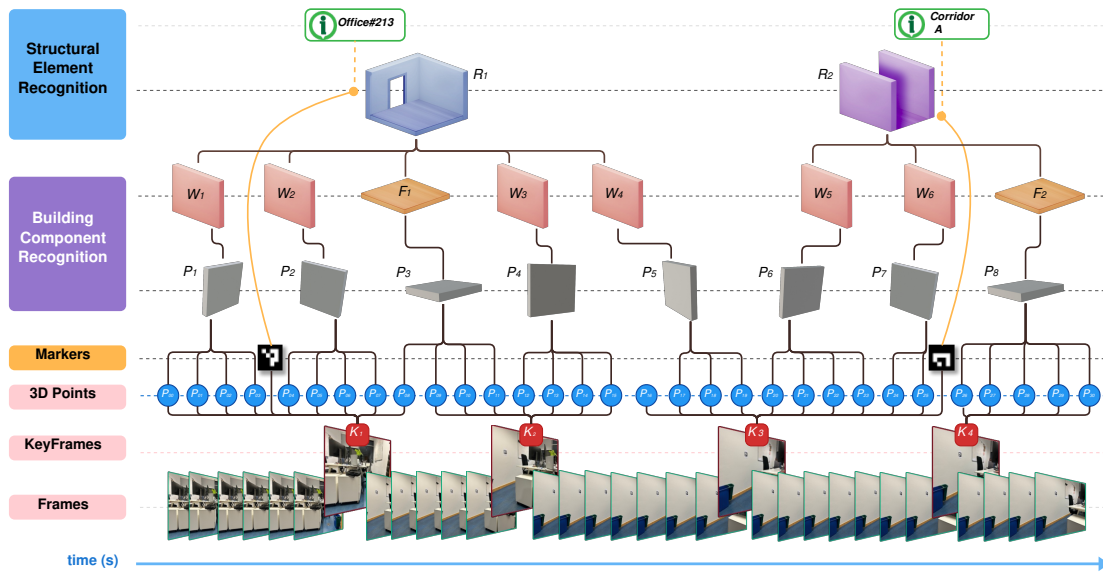


Figure 9: Scene graph structure generated using vS-Graphs, creating a hierarchical representation of the environment.



### Appendix III. Generated Scene Graphs (Qualitative Analysis)

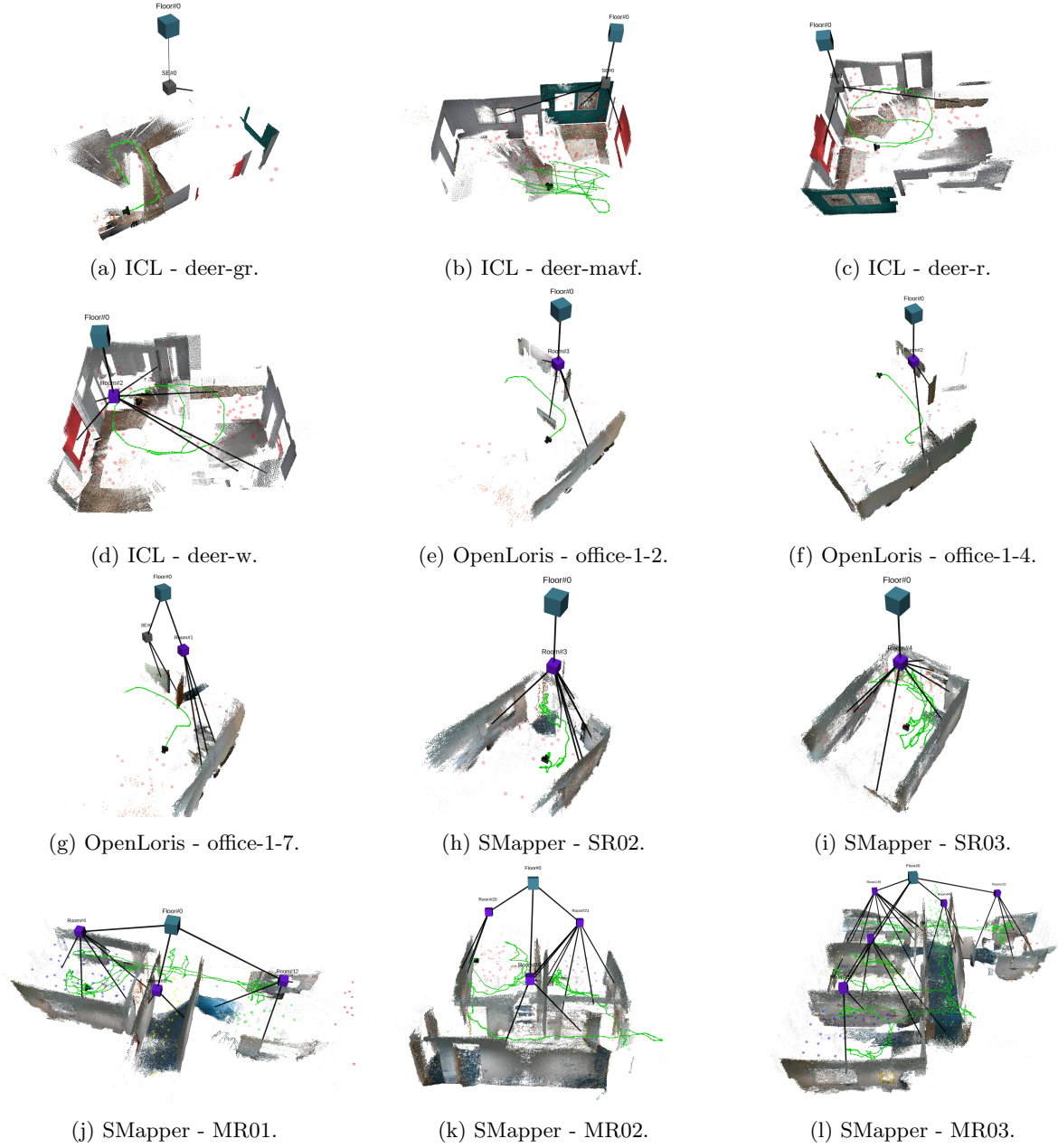


Figure 10: Scene graph examples generated by vS-Graphs across different environments, demonstrating consistent hierarchical semantic representation of structural components.

## References

- [1] H. Bavle, J. L. Sanchez-Lopez, C. Cimorelli, A. Tourani, and H. Voos, “From slam to situational awareness: Challenges and survey,” *Sensors*, vol. 23, no. 10, p. 4849, 2023.
- [2] H. Pu, J. Luo, G. Wang, T. Huang, and H. Liu, “Visual slam integration with semantic segmentation and deep learning: A review,” *IEEE Sensors Journal*, 2023.
- [3] D. Cai, R. Li, Z. Hu, J. Lu, S. Li, and Y. Zhao, “A comprehensive overview of core modules in visual slam framework,” *Neurocomputing*, p. 127760, 2024.
- [4] A. Tourani, H. Bavle, J. L. Sanchez-Lopez, R. M. Salinas, and H. Voos, “Marker-based visual slam leveraging hierarchical representations,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 3461–3467.
- [5] A. Tourani, H. Bavle, D. I. Avşar, J. L. Sanchez-Lopez, R. Munoz-Salinas, and H. Voos, “Vision-based situational graphs exploiting fiducial markers for the integration of semantic entities,” *Robotics*, vol. 13, no. 7, p. 106, 2024.
- [6] A. Rosinol, A. Gupta, M. Abate, J. Shi, and L. Carlone, “3d dynamic scene graphs: Actionable spatial perception with places, objects, and humans,” 2020.
- [7] S. Koch, P. Hermosilla, N. Vaskevicius, M. Colosi, and T. Ropinski, “Sgrec3d: Self-supervised 3d scene graph learning via object-level scene reconstruction,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 3404–3414.
- [8] S.-C. Wu, J. Wald, K. Tateno, N. Navab, and F. Tombari, “Scenegrphfusion: Incremental 3d scene graph prediction from rgb-d sequences,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 7515–7525.
- [9] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A real-time spatial perception system for 3d scene graph construction and optimization,” 2022.
- [10] A. Werby, C. Huang, M. Büchner, A. Valada, and W. Burgard, “Hierarchical open-vocabulary 3d scene graphs for language-grounded robot navigation,” in *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*, 2024.
- [11] H. Bavle, J. L. Sanchez-Lopez, M. Shaheer, J. Civera, and H. Voos, “S-graphs+: Real-time localization and mapping leveraging hierarchical representations,” *IEEE Robotics and Automation Letters*, vol. 8, no. 8, pp. 4927–4934, 2023.
- [12] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, “Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [13] A. Tourani, H. Bavle, J. L. Sanchez-Lopez, and H. Voos, “Visual slam: What are the current trends and what to expect?” *Sensors*, vol. 22, no. 23, p. 9297, 2022.
- [14] T. Whelan, S. Leutenegger, R. F. Salas-Moreno, B. Glocker, and A. J. Davison, “Elasticfusion: Dense slam without a pose graph,” in *Robotics: science and systems*, vol. 11. Rome, Italy, 2015, p. 3.
- [15] T. Schops, T. Sattler, and M. Pollefeys, “Bad slam: Bundle adjusted direct rgb-d slam,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 134–144.
- [16] C. Yan, D. Qu, D. Xu, B. Zhao, Z. Wang, D. Wang, and X. Li, “Gs-slam: Dense visual slam with 3d gaussian splatting,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 19 595–19 604.
- [17] Z. Peng, T. Shao, Y. Liu, J. Zhou, Y. Yang, J. Wang, and K. Zhou, “Rtg-slam: Real-time 3d reconstruction at scale using gaussian splatting,” in *ACM SIGGRAPH 2024 Conference Papers*, 2024, pp. 1–11.

- [18] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, “Splatam: Splat track & map 3d gaussians for dense rgb-d slam,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 21 357–21 366.
- [19] M. Grinvald, F. Furrer, T. Novkovic, J. J. Chung, C. Cadena, R. Siegwart, and J. Nieto, “Volumetric instance-aware semantic mapping and 3d object discovery,” IEEE Robotics and Automation Letters, vol. 4, no. 3, pp. 3037–3044, 2019.
- [20] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, “Nice-slam: Neural implicit scalable encoding for slam,” in Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2022, pp. 12 786–12 796.
- [21] H. Zhai, G. Huang, Q. Hu, G. Li, H. Bao, and G. Zhang, “Nis-slam: Neural implicit semantic rgb-d slam for 3d consistent scene understanding,” IEEE Transactions on Visualization and Computer Graphics, 2024.
- [22] Y. Peng, R. Xv, W. Lu, X. Wu, Y. Xv, Y. Wu, and Q. Chen, “A high-precision dynamic rgb-d slam algorithm for environments with potential semantic segmentation network failures,” Measurement, p. 118090, 2025.
- [23] Z. Teed and J. Deng, “Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras,” Advances in neural information processing systems, vol. 34, pp. 16 558–16 569, 2021.
- [24] J. He, M. Li, Y. Wang, and H. Wang, “Ovd-slam: An online visual slam for dynamic environments,” IEEE Sensors Journal, vol. 23, no. 12, pp. 13 210–13 219, 2023.
- [25] P. Cong, J. Liu, J. Li, Y. Xiao, X. Chen, X. Feng, and X. Zhang, “Ydd-slam: Indoor dynamic visual slam fusing yolov5 with depth information,” Sensors, vol. 23, no. 23, p. 9592, 2023.
- [26] J. Han, R. Dong, and J. Kan, “Basl-ad slam: A robust deep-learning feature-based visual slam system with adaptive motion model,” IEEE Transactions on Intelligent Transportation Systems, 2024.
- [27] Z. Zhang, Y. Song, B. Pang, X. Yuan, Q. Xu, and X. Xu, “Ssf-slam: Real-time rgb-d visual slam for complex dynamic environments based on semantic and scene flow geometric information,” IEEE Transactions on Instrumentation and Measurement, 2025.
- [28] Y. Li, H. Zhao, X. Qi, L. Wang, Z. Li, J. Sun, and J. Jia, “Fully convolutional networks for panoptic segmentation,” in Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2021, pp. 214–223.
- [29] J. Hu, L. Huang, T. Ren, S. Zhang, R. Ji, and L. Cao, “You only segment once: Towards real-time panoptic segmentation,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 17 819–17 829.
- [30] K. G. Derpanis, “Overview of the ransac algorithm,” Image Rochester NY, vol. 4, no. 1, pp. 2–3, 2010.
- [31] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto, “Voxblox: Incremental 3d euclidean signed distance fields for on-board mav planning,” in IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2017.
- [32] H. Bavle, J. L. Sanchez-Lopez, M. Shaheer, J. Civera, and H. Voos, “Situational graphs for robot navigation in structured indoor environments,” IEEE Robotics and Automation Letters, vol. 7, no. 4, pp. 9107–9114, 2022.
- [33] S. Saeedi, E. D. Carvalho, W. Li, D. Tzoumanikas, S. Leutenegger, P. H. Kelly, and A. J. Davison, “Characterizing visual localization and mapping datasets,” in 2019 International Conference on Robotics and Automation (ICRA). IEEE, 2019, pp. 6699–6705.

- [34] X. Shi, D. Li, P. Zhao, Q. Tian, Y. Tian, Q. Long, C. Zhu, J. Song, F. Qiao, L. Song et al., “Are we ready for service robots? the openloris-scene datasets for lifelong slam,” in 2020 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2020, pp. 3139–3145.
- [35] A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner, “Scannet: Richly-annotated 3d reconstructions of indoor scenes,” in Proceedings of the IEEE conference on computer vision and pattern recognition, 2017, pp. 5828–5839.
- [36] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, “A benchmark for the evaluation of rgb-d slam systems,” in 2012 IEEE/RSJ international conference on intelligent robots and systems. IEEE, 2012, pp. 573–580.
- [37] P. M. B. Soares, A. Tourani, M. Fernandez-Cortizas, A. Bikandi-Noya, H. Voos, and J. L. Sanchez-Lopez, “Smapper: A multi-modal data acquisition platform for slam benchmarking,” Journal of Intelligent & Robotic Systems, vol. 112, no. 20, 2026.
- [38] F. J. Romero-Ramirez, R. Muñoz-Salinas, M. J. Marín-Jiménez, M. Cazorla, and R. Medina-Carnicer, “sslam: Speeded-up visual slam mixing artificial markers and temporary keypoints,” Sensors, vol. 23, no. 4, p. 2210, 2023.
- [39] R. Muñoz-Salinas and R. Medina-Carnicer, “Ucoslam: Simultaneous localization and mapping by fusion of keypoints and squared planar markers,” Pattern Recognition, vol. 101, p. 107193, 2020.